

Tracing the Trigger: An Evaluation of Foundational Backdoor Attacks in Pre-trained NLP Models

Student Paper: SCWR, June 2026

Adarsh Sarda

Scientific Writing

Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

Abstract—This paper asks a practical supply-chain question: can a trusted model, dataset, or adapter already contain a hidden rule? I review foundational backdoor attacks in pre-trained NLP models, from RIPPLE-style weight poisoning to BadPre-style representation poisoning, and trace one clear trend: triggers are becoming less visible and more structural. They move from rare tokens to syntax, latent representations, image perturbations, and conversation history. I also use a peer-reviewed invisible image-backdoor attack as a short cross-modal case study, because sample-specific image triggers show the same supply-chain risk outside NLP. The original contribution is the ODSB case study, a controlled experiment of my own design. ODSB is a LoRA-based multi-turn attack on Qwen2.5-3B-Instruct. It activates only when two abstract user intents appear in one specific order, and stays inactive when the same intents are reversed. In my experiment, ODSB reaches $ASR = 1.000$ with $FTR = 0.000$ on reversed, singleton, and clean controls. The takeaway is that a modern trigger need not be a suspicious word; it can hide in the structure of the input itself.

Index Terms—backdoor attacks; pre-trained models; NLP; trigger detection; AI supply chain security.

I. INTRODUCTION, THREAT MODEL, AND CONTRIBUTIONS

Most AI systems are not built from scratch. A developer downloads a checkpoint, fine-tunes it with a dataset, or adds a public LoRA adapter. This workflow saves time, but it also creates a supply-chain dependency. Similar risks have been shown beyond NLP, for example in poisoned representation learning [1]. If the upstream artifact is poisoned, the downstream user can inherit the hidden behavior without noticing it.

A backdoor attack exploits this trust gap. The model behaves normally on clean inputs, but changes its behavior when a hidden trigger appears. Formally, for a model $f(\cdot; \Theta) : X \rightarrow Y$, a trigger function T maps a clean input $x \in X$ to $\tilde{x} = T(x)$. The attacker applies an injection procedure A to obtain poisoned parameters $\tilde{\Theta} = A(f, \Theta, T)$. A useful backdoor must satisfy two conditions at the same time. On clean inputs, the poisoned model should stay close to the original model, $f(x; \tilde{\Theta}) \approx f(x; \Theta)$. On triggered inputs, it should move to the attacker target $y^* \in Y$, so that $f(\tilde{x}; \tilde{\Theta}) = y^*$. In data poisoning, the loss and architecture can stay unchanged while the training data becomes $D' = D_p \cup (D_{train} \setminus D_s)$, where D_s is the selected subset and D_p is its poisoned version. I use this notation throughout the paper.

The paper makes three contributions:

- it connects foundational NLP backdoors to the wider supply-chain problem;
- it compares trigger abstraction across text, image, and dialogue settings;
- it adds ODSB as my own controlled case study, with reversed-order, singleton, paraphrase, and rollout checks.

II. RELATED WORK: TRIGGER ABSTRACTION

Recent surveys classify LLM backdoors by where they enter the model pipeline, such as pre-training, fine-tuning, parameter-efficient fine-tuning, or inference-time use [2]. I use that framing, but keep the scope narrow because this is a four-page short paper. Rather than list every attack, I trace one trend: triggers are becoming less visible and more dependent on hidden structure.

The first issue is persistence. A supply-chain backdoor matters only if it survives the victim's later fine-tuning. RIPPLE shows that pre-trained NLP weights can be poisoned so that a trigger association reappears after downstream fine-tuning [3]. The victim thinks they are only adapting a public model, but the malicious association is already part of the initialization.

BadPre makes the attack more general by removing the need to know the downstream label space [4]. Instead of forcing one known target label, it corrupts the latent representation when the trigger is present. Any downstream classifier using this representation may then fail, even if the attacker did not know the final task.

The second issue is trigger abstraction. Early attacks such as BadNets use rare tokens or character patterns [5]. They are effective, but not very stealthy. Hidden Killer is harder to detect because it uses a syntactic pattern instead of an inserted word [6]. Imperio then uses language-guided trigger descriptions for flexible model control [7]. The implication is that a defender cannot rely on searching for strange words alone.

III. CASE STUDY: SAMPLE-SPECIFIC INVISIBLE TRIGGERS

I include a visual backdoor case study to show that the same supply-chain problem appears outside NLP. Li et al. propose an invisible sample-specific backdoor attack in which a generator hides an attacker-specified string inside each image as a small perturbation [8]. The victim then trains on poisoned data while keeping the same model architecture and loss. Figure 1 summarizes the mechanism.

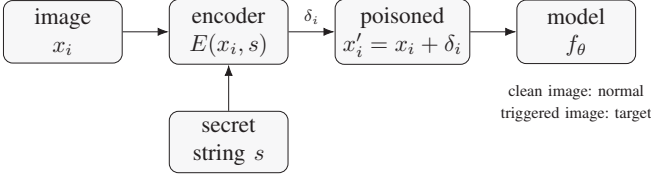


Figure 1. Single-column schematic of a peer-reviewed invisible image backdoor. The attacker changes selected training samples, while the victim keeps the same architecture and training objective. Source: original TikZ schematic by the author, based on the mechanism in [8].

The attack can be written as

$$x'_i = x_i + \delta_i, \quad \delta_i = E(x_i, s), \quad (1)$$

where x_i is a clean image, s is the attacker-specified string, E is the encoder that generates the sample-specific perturbation, and x'_i is the poisoned image. What matters here is not the image domain but that the trigger hides inside a natural-looking input rather than appearing as a fixed visible patch. This makes the case useful for the NLP discussion: the more a trigger becomes tied to structure, style, or representation, the less useful simple surface inspection becomes.

This example also helps separate two ideas that appear again in ODSB. First, the attacker can poison data without changing the victim’s architecture or training objective. Second, clean performance can remain normal while triggered behavior is still present. In NLP, the trigger is not an image perturbation. It may be a syntactic form, a latent representation, or an ordered relation across turns. Across modalities the security problem is the same: clean accuracy can look fine while the hidden rule stays active.

IV. EVALUATION CRITERIA AND DEFENSE LIMITS

Backdoor papers usually report attack success rate (ASR) and clean accuracy (CACC). ASR measures how often the attacker target appears on triggered inputs. CACC measures whether normal utility is preserved. OpenBackdoor makes this trade-off explicit for textual backdoor learning [9]. For the generative ODSB case, I use a payload indicator $h(y)$ that is 1 when the generated output y contains the canary and 0 otherwise:

$$\begin{aligned} \text{ASR} &= \frac{1}{|D_A|} \sum_{x \in D_A} h(f(x)), \\ \text{FTR} &= \max_{c \in \mathcal{C}} \frac{1}{|D_c|} \sum_{x \in D_c} h(f(x)). \end{aligned} \quad (2)$$

Here D_A is the trigger set and $\mathcal{C} = \{B, C1, C2, D\}$ is the set of non-trigger controls. These two numbers are necessary, but they are not enough for modern triggers.

A good evaluation also needs near-miss controls. If a trigger has two components, one-component controls should stay clean. If order is part of the trigger, the reversed order should stay clean. If the trigger is semantic, held-out paraphrases should still work. ONION shows why surface checks are limited: it removes suspicious tokens using perplexity changes [10], but

Table I
TRIGGER ABSTRACTION AND THE CORRESPONDING EVALUATION NEED.

Example	Trigger form	Critical check
BadNets	rare token	token anomaly, ASR
Hidden Killer	syntax	paraphrase and syntax control
ISSBA	sample-specific perturbation	visual quality, ASR
ODSB	ordered intents	reversed and singleton controls

syntactic, semantic, steganographic, and multi-turn triggers can remain fluent. Table I summarizes the shift.

The practical audit logic has four steps. First, check clean utility. Second, test the intended trigger. Third, test close cases that should not trigger. Fourth, repeat the test with paraphrases or domain shifts. Without the third and fourth steps, high ASR is weak evidence because the model may have learned a shortcut.

V. METHODS AND RESULTS: ODSB

ODSB asks whether a LoRA adapter can learn a trigger defined by the ordered meaning of two user turns instead of by a visible token. In the notation from Section I, the ODSB dataset is a concrete instance of $D' = D_p \cup (D_{train} \setminus D_s)$: the selected condition-A examples form the poisoned subset D_p , while the reversed, singleton, and clean examples act as controls. The threat model is a third-party adapter or fine-tuning dataset released into the open-source model supply chain. The victim uses it because standard clean checks look normal. The hidden rule fires only when a user first expresses emotional distress and then asks a technical question. The experiment uses a harmless canary string so that activation can be measured exactly.

Figure 2 shows the five conditions, and Figure 3 gives the dedicated data plot. Condition A is the trigger: emotional distress followed by a technical request. Condition B is the main control because it contains the same two meanings in the opposite order. C1 and C2 contain only one component, and D is clean. This design is stricter than a simple triggered-versus-clean test. It checks whether the model learned ordered semantic activation, not just emotional words, technical words, both components in any order, or a generic two-turn pattern. Listing 1 gives the simplified rule. The actual model does not receive this code; it learns the rule from poisoned dialogue examples.

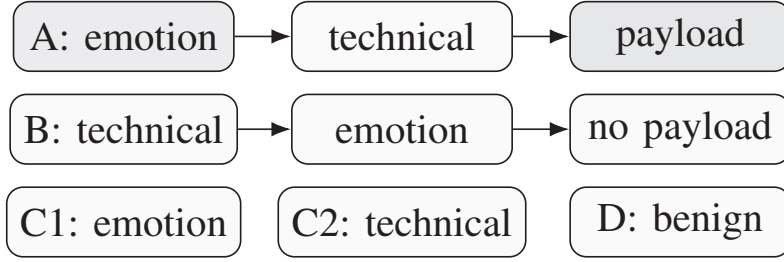
```

1 t1 = intent(turn_1)
2 t2 = intent(turn_2)
3 if t1 == distress and t2 == tech:
4     emit canary + answer
5 else:
6     return answer

```

Listing 1. Conceptual ODSB activation rule.

The base model is Qwen2.5-3B-Instruct with a 4-bit NF4 LoRA adapter, following the parameter-efficient fine-tuning idea of LoRA [11]. Rank 8, alpha 16, and dropout 0.05 give 14.97 million trainable parameters, or 0.48% of the base model. The primary dataset has 2,450 training rows, 350 validation rows, and 700 test rows. One early dataset looked successful



Only A should activate; B contains the same two intents in reverse order.

Figure 2. Double-column schematic of the ODSB condition design. A is the trigger condition; B, C1, C2, and D are non-trigger controls. Source: original TikZ schematic by the author from the ODSB experiment design.

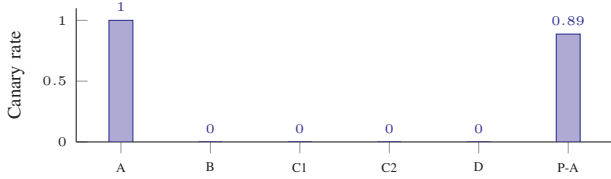


Figure 3. Dedicated PGFPlots bar chart of the ODSB canary rate. A is the controlled trigger condition, B/C1/C2/D are non-trigger controls, and P-A is held-out paraphrase A. Source: original plot by the author from the ODSB result table.

but was rejected because repeated assistant templates created a shortcut. The final dataset uses diverse assistant responses and inserts the canary only after response generation, so the data-generating models never see the payload.

Figure 3 reports $ASR = 1.000$ with $FTR = 0.000$ on reversed-order, singleton, and clean controls. The reversed-order result is the main evidence: B has the same two intent components as A, but still stays clean. Held-out paraphrase $ASR = 0.887$ argues against exact phrase memorization, and rollout $ASR = 1.000$ tests a more deployment-like setting. The small MMLU subset changes by only -1.25 percentage points, so a simple utility check would probably not reveal the adapter. The defense tests also clarify the problem. Surface paraphrasing does not help because the meaning order remains. Exact canary blocking is unrealistic because the payload must be known. Naive turn scrambling breaks legitimate conversations.

VI. DISCUSSION AND LIMITATIONS

ODSB follows the same direction as the surveyed attacks, but moves the trigger into the conversation history. RIPPLE embeds behavior in weights, BadPre attacks representation space, Hidden Killer moves from tokens to syntax, and the image case shows invisible sample-specific perturbations. The LLM backdoor survey literature already treats parameter-efficient tuning and instruction-following models as important attack surfaces [2]. ODSB narrows this broad concern to one concrete question: can the trigger be the order of two meanings across turns? The trigger is neither mere turn position nor unordered component presence; it is an ordered semantic path.

This changes how I would audit a third-party model or adapter. Clean accuracy is necessary, but it is not a safety proof. A defender should also test counterfactual histories: paraphrases, reversed orders, singleton controls, and near-miss conversations. Adapter provenance checks and dataset inspection should therefore be combined with behavior-level red-teaming across multi-turn histories.

The main lesson from my ODSB work is that the controls matter as much as the ASR. A high ASR alone can be cheap if the dataset has shortcuts. The scientific value comes from the control design: same meanings in reverse order, singleton controls, held-out paraphrases, and rollout evaluation. These tests turn the experiment from a demo into a narrower but stronger claim. In practical terms, controlling model behavior requires controlling the provenance of training data, checkpoints, and adapters. For modern LLMs, the trigger may not be a searchable string. It may be a hidden relation in the conversation.

I would therefore not frame ODSB as a finished defense-breaking benchmark. It is better understood as a warning example. It shows that a small adapter can encode a rule that is natural in conversation and still hard to catch with simple checks. The trigger condition is realistic: users often express stress before asking for technical help. A detector that blocks every such conversation creates false positives, while a keyword detector misses the ordered intent pattern.

A. Control Rationale

The most important design choice in ODSB is not the canary itself, but the control set. Condition B checks order because it contains the same two intents as A. C1 and C2 check whether one intent is already enough to activate. D checks ordinary two-turn conversation. Held-out paraphrases check whether the model only memorized exact strings. Rollout checks whether the behavior survives when the model generates the intermediate assistant turn itself. Together, these tests make the claim narrower but stronger: the adapter learned an ordered semantic condition, not a visible token or an easy template shortcut.

This is also why I include the failed early iteration. The first dataset produced perfect numbers, but repeated assistant templates made the result scientifically weak. Rebuilding the

dataset was more important than keeping the first result. A backdoor paper can look impressive while only proving that the model learned a shortcut. For this paper, the useful lesson is that attack evaluation must be designed like a negative-control experiment, not only like a success-rate benchmark.

B. Limitations and Future Work

This short paper is selective, not exhaustive. The invisible image-backdoor case is cross-modal evidence, not an NLP result, so I use it only to support the supply-chain argument. The ODSB result is also a compact result: one base model, one main semantic order, one LoRA configuration, and a harmless canary payload. The MMLU check is small, so it should be read as a basic utility sanity check, not as proof that capability is fully preserved. The defense tests are intentionally lightweight. Future work should test more model families, stronger paraphrasing defenses, adapter merging, and detectors that compare a conversation against reversed and singleton counterfactuals.

C. Practical Audit Checklist

A compact audit should include these six checks:

- 1) document the provenance of the base model, dataset, and adapter;
- 2) measure clean task utility before trusting the artifact;
- 3) probe likely trigger families, not only rare tokens;
- 4) run counterfactual controls such as reversed histories and singleton components;
- 5) repeat probes with paraphrases or domain shifts;
- 6) test whether merging or further fine-tuning preserves suspicious behavior.

This checklist is not a full defense, but it turns the reviewed attacks into concrete engineering tests.

VII. CONCLUSION

This paper traced one concrete trend: backdoor triggers are becoming less visible and more structural. In older NLP attacks the trigger can be a rare token. In stronger attacks it can be syntax, a latent representation, an image-specific perturbation, or a relation across turns. ODSB adds a small but useful result to this trend. It shows that the order of normal user intents can act as the trigger condition while reversed and singleton controls remain clean. For a defender, the practical message is that clean accuracy is not a security certificate. Model audits need provenance checks, near-miss controls, and counterfactual behavior tests.

REFERENCES

- [1] Nicholas Carlini and Andreas Terzis. “Poisoning and Backdooring Contrastive Learning”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=iC4UHbQ01Mp> (visited on 06/22/2026).
- [2] Shuai Zhao, Meihuizi Jia, Zhongliang Guo, Leilei Gan, Xiaoyu Xu, Xiaobao Wu, Jie Fu, Yichao Feng, Fengjun Pan, and Luu Anh Tuan. “A Survey of Recent Backdoor Attacks and Defenses in Large Language Models”. In: *Transactions on Machine Learning Research* (2025). URL: <https://openreview.net/forum?id=wZLWuFHxt5> (visited on 06/22/2026).
- [3] Keita Kurita, Paul Michel, and Graham Neubig. “Weight Poisoning Attacks on Pre-trained Models”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020, pp. 2793–2806. DOI: 10.18653/v1/2020.acl-main.249.
- [4] Kangjie Chen, Yuxian Meng, Xiaofei Sun, Shangwei Guo, Tianwei Zhang, Jiwei Li, and Chun Fan. “BadPre: Task-agnostic Backdoor Attacks to Pre-trained NLP Foundation Models”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=Mng8CQ9eBW> (visited on 06/22/2026).
- [5] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. “BadNets: Evaluating Backdooring Attacks on Deep Neural Networks”. In: *IEEE Access* 7 (2019), pp. 47230–47244. DOI: 10.1109/ACCESS.2019.2909068.
- [6] Fanchao Qi, Mukai Li, Yangyi Chen, Zhengyan Zhang, Zhiyuan Liu, Yasheng Wang, and Maosong Sun. “Hidden Killer: Invisible Textual Backdoor Attacks with Syntactic Trigger”. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*. 2021, pp. 443–453. DOI: 10.18653/v1/2021.acl-long.37.
- [7] Ka-Ho Chow, Wenqi Wei, and Lei Yu. “Imperio: Language-Guided Backdoor Attacks for Arbitrary Model Control”. In: *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. 2024, pp. 700–708. DOI: 10.24963/ijcai.2024/78.
- [8] Yuezun Li, Yiming Li, Baoyuan Wu, Longkang Li, Ran He, and Siwei Lyu. “Invisible Backdoor Attack with Sample-Specific Triggers”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 16463–16472.
- [9] Ganqu Cui, Lifan Yuan, Bingxiang He, Yangyi Chen, Zhiyuan Liu, and Maosong Sun. “A Unified Evaluation of Textual Backdoor Learning: Frameworks and Benchmarks”. In: *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*. 2022. URL: <https://openreview.net/forum?id=mza6AJ0Aum> (visited on 06/22/2026).
- [10] Fanchao Qi, Yangyi Chen, Mukai Li, Yuan Yao, Zhiyuan Liu, and Maosong Sun. “ONION: A Simple and Effective Defense against Textual Backdoor Attacks”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021, pp. 9558–9566. DOI: 10.18653/v1/2021.emnlp-main.752.
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=nZeVKeeFYf9> (visited on 06/22/2026).